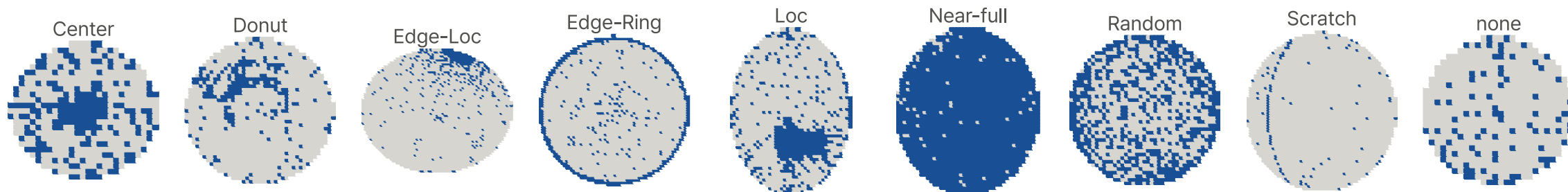


웨이퍼 맵 불량 패턴 분류

lot 단위로 나눈 평가에서 macro-F1 0.909를 확인한 소형 CNN

WM-811K 공개 데이터



이진성

01 프로젝트 개요

학습에 쓰지 않은 lot 25,444장에서 macro-F1 0.909. 평가 방식을 먼저 고치고 다섯 가지로 검증

데이터	WM-811K 웨이퍼 맵 811,457장 중 라벨 있는 172,950장, 불량 패턴 9종
문제	같은 웨이퍼의 복사본이 학습과 평가에 섞이는 누수를 막고, 새 lot에서 성능 재기
결과	macro-F1 0.843(베이스라인) → 0.909(CNN), 모든 패턴 recall 0.77 이상
AI 활용	구현·실험 Claude Code / 판단·검증 본인 / 별도 검증 에이전트 45개. 체크포인트 6개에서 판단 13건, 오류 15건 기록

용어 lot: 함께 처리한 웨이퍼 묶음(최대 25장) · macro-F1: 패턴 9종 F1의 평균 · recall: 실제 그 패턴 중 맞힌 비율

진행 순서

1 문제 정의와 평가 설계	3쪽
2 베이스라인과 소형 CNN	4쪽
3 검증과 오류 수정	5쪽
4 AI 활용 ① 시간과 자원	6쪽
5 AI 활용 ② 지시와 판단	7쪽
6 AI 활용 ③ 검증과 오류 적발	8쪽
7 결론·한계·재현 정보	9쪽

02 문제 정의: 데이터와 평가 방식

에이전트 10개가 찾은 이상점 17개 중 복사본 누수를 핵심 문제로 선정. lot과 복사본을 한 묶음으로 나누는 규칙 수립

누수 3겹

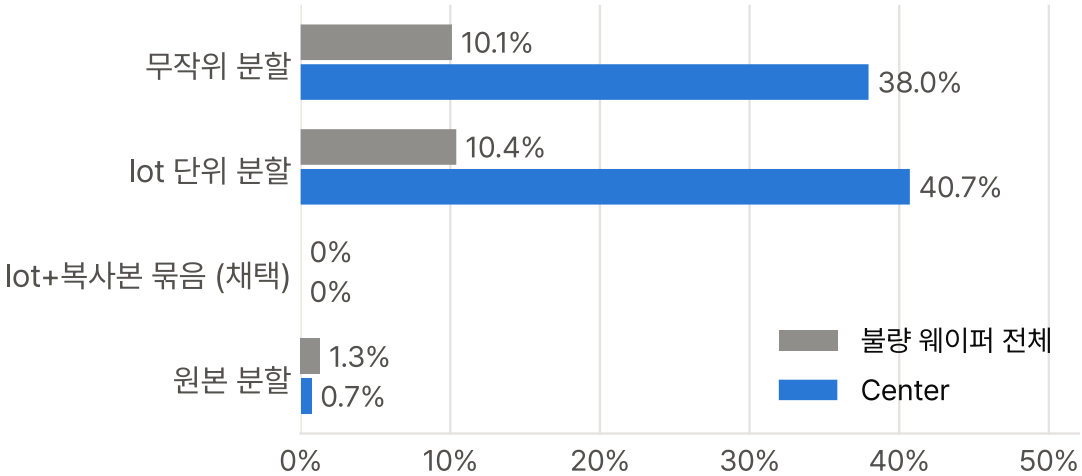
원본 분할	test 3,158장이 train의 복사본
lot 단위 분할	재검사 복사본이 양쪽에 남음(Center 40.7%)
반전·회전	뒤집거나 돌린 복사본까지 8가지 변환으로 비교

채택한 분할 lot + 10픽셀 이내 복사본을 한 묶음으로 70:15:15

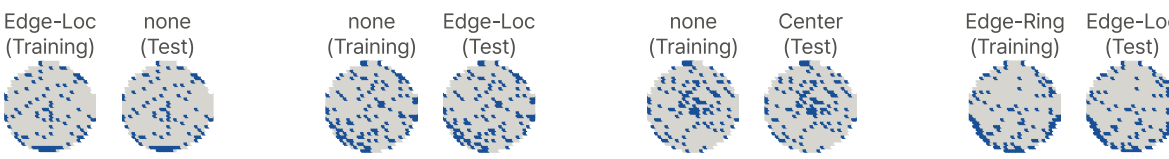
성공 기준 macro-F1 0.80 이상,
모든 패턴 recall 0.50 이상, 베이스라인 초과

Claude Code	탐색·분할 스크립트 구현, 이상점 탐색 에이전트 10개 병렬 실행
본인	정제 규칙·분할 방식·성공 기준 결정(판단 지점 6·7)

test 웨이퍼 중 학습 쪽에 10픽셀 이내 복사본이 있는 비율



같은 맵, 다른 라벨: 원본 Training/Test 사이 복사본 14쌍 중 4쌍



03 방법과 결과

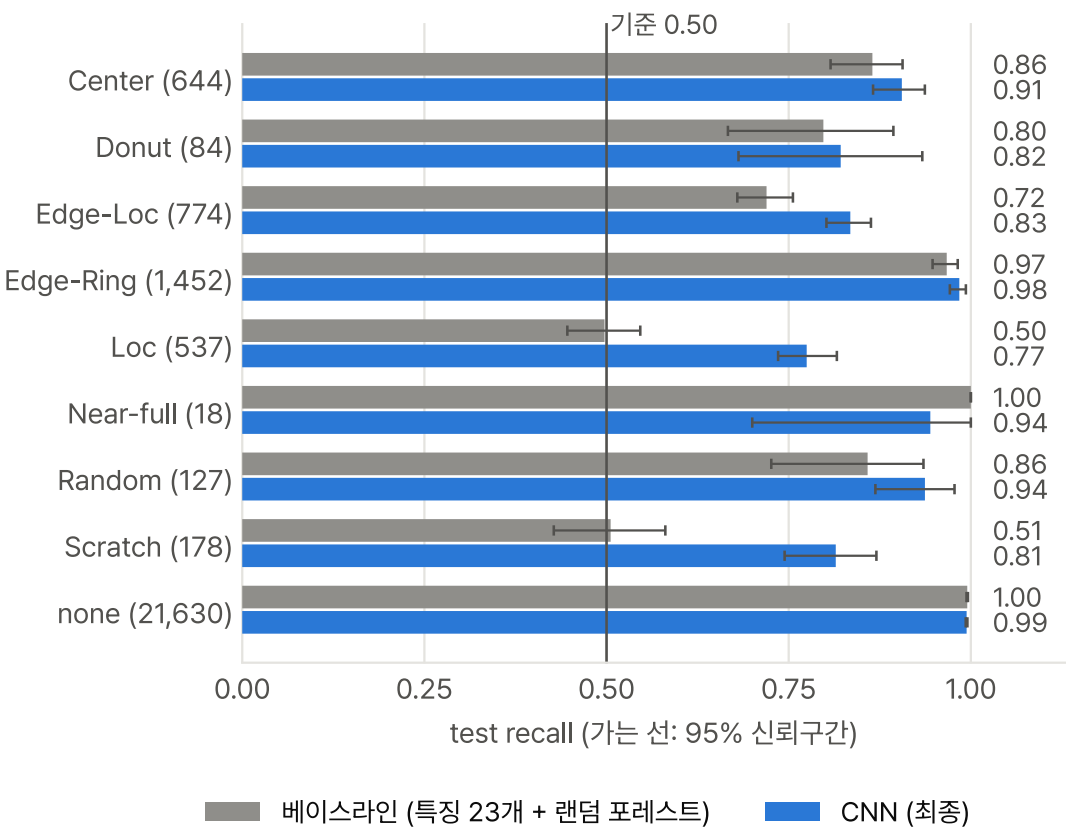
같은 test 25,444장에서 소형 CNN macro-F1 0.843 → 0.909. Loc-Scratch recall이 가장 크게 상승

모델	macro-F1 (95% CI)	최저 recall
베이스라인 (특징 23개 + RF)	0.843 (0.820–0.858)	0.497 (Loc)
소형 CNN	0.901 (0.863–0.918)	0.763 (Loc)
소형 CNN + val 보정 (최종)	0.909 (0.885–0.922)	0.775 (Loc)

방법 64×64 축소(블록 최대값으로 Scratch 보존),
반전·회전 증강, 합성곱 4블록 58만 파라미터.
보정값은 val에서만 정함

효과 없던 시도 class-weighted CE: 0.825,
none 오탐 증가 → 미채택

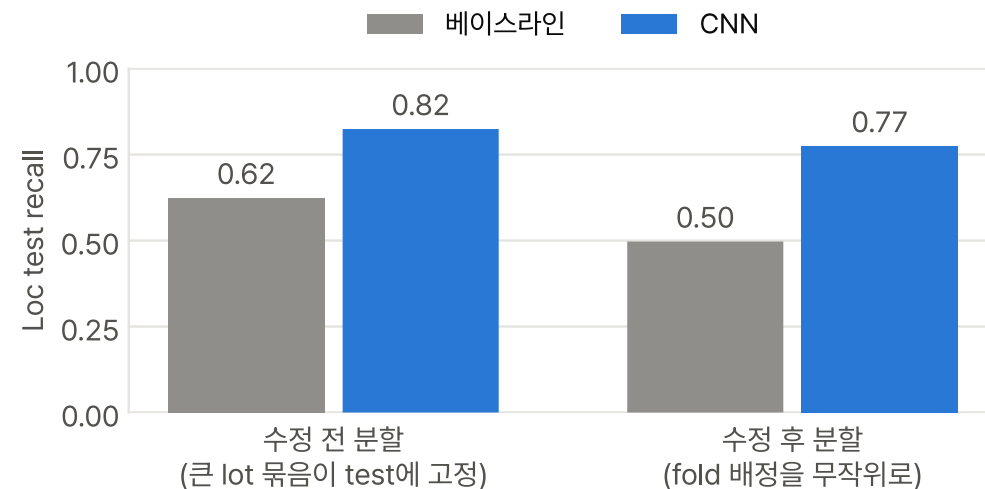
Claude Code 특징 23개·RF·CNN 구현, 학습 5회
본인 개선 방향과 입력 크기 결정(판단 지점 8),
weighted CE 폐기(판단 지점 10)



예상보다 높은 점수 → 검증 에이전트 4개와 개입 실험 → 분할 절차 오류 1건 발견·수정

검증	방법	결과
우연 수준	라벨을 섞어 학습	macro-F1 0.102
누수 제거 효과	무작위 분할 vs lot 분할	0.895 vs 0.909
지름길 학습	제품 외곽선을 바꿔 끼움	예측 99.5% 유지
오분류 검토	96장, 라벨 가린 독립 판정	라벨 의심 27%
재현	새 폴더에서 처음부터 재실행	10개 항목 일치

Claude Code 검증 에이전트 4개 병렬, 개입 실험 구현
본인 오류 판정, 전부 재학습 결정(판단 지점 9)

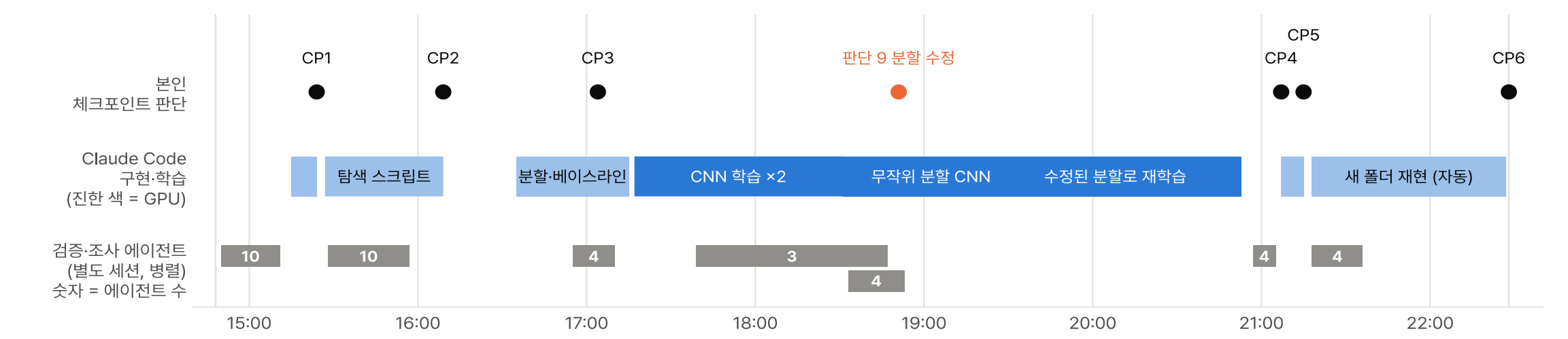


분할 절차 오류 (LOG E11)

- 원인** StratifiedGroupKFold가 큰 그룹을 seed와 무관하게 같은 fold에 고정 → 쉬운 Loc 132장이 항상 test
- 발견** 베이스라인이 이유 없이 오름 → seed 10개로 재분할 → lot 묶음 하나를 빼 보는 실험
- 수정** fold 배정 무작위 치환, 전 모델 재학습(0.893 → 0.909), 분할 후 겹침 검사 추가

05 AI 활용 ① 시간과 자원

7시간 40분에 계획부터 재현 확인까지. 학습이 도는 동안 검증·조사 에이전트 35개를 병렬 운용



총 작업 시간	7시간 40분 (14:48–22:28), 1인
GPU 학습	5회, 3시간 36분 (Quadro P2000)
코드	스크립트 12개 1,766줄, 커밋 16회
에이전트	워크플로 8회 35개, 병렬 실행 약 3시간
기록	LOG 1,155줄, PLAN·RESULT·GLOSSARY 412줄

- 병렬**
CNN 학습이 도는 동안 검증 에이전트 4개가 지표 재계산·짝비교·지름길·이중맹검 수행(18:33–18:53)
- 탐색**
에이전트 10개가 29분에 이상점 17개 → 스크립트로 재검산 1분 39초
- 재현**
새 폴더 재실행 1시간 8분은 스크립트가 자동 비교(10개 항목)

06 AI 활용 ② 지시와 판단

첫 지시문에 멈출 지점과 기록 규칙을 명시. 체크포인트 6개에서 판단 13건

첫 지시문의 규칙 (원문)

“판단이 필요한 지점에서는 멈추고 나에게 묻는다. 선택지 2~3개와 각각의 장단점을 제시하고, 추천이 있으면 이유와 함께 밝힌다.”

“숫자는 실행 결과에서만 가져온다. 추정치를 결과처럼 쓰지 않는다.”

“오류 기록: 네 코드나 판단이 틀렸던 경우, 누가(나/너) 어떻게 발견했고 무엇을 바꿨는지.”

작업 흐름

본인 문제 정의 · 성공 기준 · 지시문



Claude Code 구현 · 실험 · 선택지 제시



검증 에이전트 재계산 · 반박 · 개입 실험



본인 체크포인트에서 판단 · 기록

체크포인트 3: AI의 제안과 내 결정

AI가 보고한 개선 방향 후보	내 결정 (판단 지점 8)
① 소형 CNN: 맵을 격자로 맞추고 회전·반전 증강. 손으로 만든 특징은 모양·위치 정보를 뭉갬. ② 특징 강화 + 부스팅: 가볍지만 개선 폭 제한. ③ 사후 임계값 보정: 빠르지만 드문 패턴은 val 과적합 위험	①+③ 조합. weighted CE는 같은 test에서 비교해 결정하고 보정값은 val에서만. 비교 기준은 test 최고 베이스라인. 결과: weighted CE 0.825로 폐기, 보정 채택 0.909

07 AI 활용 ③ 검증과 오류 적발

구현 AI와 분리한 검증 에이전트가 AI 오류 15건 중 7건 적발. 8건은 자체 점검 규칙으로 발견

AI가 낸 오류를 잡은 사례

오류	어떻게 드러났나	조치
E10 잘못된 원인 가설	AI가 짠 진단 스크립트가 자기 가설을 반증	가설 철회
E11 분할 절차 오류	반박 검증 에이전트가 sklearn 소스와 개입 실험으로 확인	절차 수정, 전부 재학습
E13 확률 열 버그	지표 검증 에이전트가 보정 결과를 재현하다 발견	저장 로직 수정

산출물 반려 2회(대시보드 문구, 포트폴리오 1차).
참고 사례를 조사하게 한 뒤 다시 씀

규칙 검증 에이전트에는 원본 코드 대신 데이터와
결과 파일만 주고 다시 계산하게 함

검증·조사 워크플로 10회 (에이전트 45개)

단계	에이전트	결과
데이터 확보·문헌 조사	10	조사 5 + 반박 5 → E1~E4
데이터 이상점 탐색	10	탐색 4 + 검증 → 정정 7건
베이스라인·분할 재계산	4	재계산, 재구현, 코드 감사
Loc recall 원인 조사	3	원인 추적·반박 → E11
CNN 검증 2회	8	지표·짜비교·지름길·이중맹검 → E13
문체·포트폴리오 조사	10	대시보드 4, 포트폴리오 6

같은 제품의 새 lot에 대한 1차 분류 보조 수준. 수율 개선 효과는 주장하지 않음

결론

- 누수를 막은 평가에서 CNN 0.909, 베이스라인 0.843
- 분할 절차 하나로 Loc recall 12%p 차이. 모델보다 평가 방식부터 확인
- 검증 5종 통과, 새 폴더 재현 일치

한계·다음 단계

- 한 번 나눈 분할. seed 10개 평균은 약 0.01 낮음 → 그룹 5-fold
- Near-full test 18장, 신뢰구간 0.70–1.00
- 짧은 Scratch 약점, 라벨 노이즈 14쌍 → 라벨 기준 재정의

재현 정보

환경	Python 3.12.6, torch 2.7.1+cu118, scikit-learn 1.9.1, Quadro P2000, 시드 0
실행	python scripts/run_all.py (약 4~5시간) → outputs/*.json
저장소	github.com/sl2ver/wm811k-wafer-defect-classification — PLAN.md / LOG.md(판단 13건·오류 15건) / RESULT.md